

OSS 링크 : <https://oss.inha.ac.kr/project/detail/1486>

1. 주제 선정 및 문제 정의

핵심 문제 : 영화 추천 알고리즘

문제의 배경, 목적, 기대효과 :

최근 영화 시장이 과거에 비해 많이 위축되고 있다. 본인 취향에 맞는 영화가 개봉했음에도, 관련 정보를 접하지 못하는 것이 그 원인 중 하나라고 생각한다.

이에 영화들에 매긴 평점 및 리뷰 데이터를 체계적으로 수집, 분석하여 사용자별 선호 장르, 평점 경향을 반영한 영화를 추천하는 데이터베이스를 구축한다. 기대효과는 사용자의 영화 리뷰와 평점을 기반으로 개인 맞춤형 영화 추천이 가능해지며, 영화 제작사 및 플랫폼 입장에서 도 각 세대, 성별 별 선호 장르, 감독, 출연진 분석이 가능해져 마케팅 전략 수립에 도움이 될 것으로 예상된다.

2. 데이터베이스 설계

[사용자(user) 테이블]

- 사용자 테이블은 user_id를 기본키(PK)로 하며, username(VARCHAR, NOT NULL), birth_year(INT), gender(CHAR(1), CHECK(gender IN ('M','F'))), signup_date DATE 속성으로 구성된다.
- 각 사용자(user)는 여러 리뷰를 작성할 수 있으므로 리뷰(review) 테이블과 1:N 관계를 가진다.
- user_id는 review 테이블의 외래키로 참조되며, 참조 무결성을 유지하기 위해 ON DELETE CASCADE 제약조건이 사용될 수 있다.

[영화(movie) 테이블]

- 영화 테이블은 movie_id를 기본키로 하며, title(VARCHAR, NOT NULL), release_year(INT), genre(VARCHAR), director(VARCHAR), runtime(INT) 속성으로 구성된다.
- 각 영화는 여러 리뷰를 받을 수 있으므로 review 테이블과 1:N 관계, 여러 배우와 출연 관계를 가지므로 casting 테이블과 N:M 관계를 가진다.
- movie_id는 review 및 casting 테이블에서 외래키로 참조되며, 참조 무결성을 보장하기 위해 ON DELETE RESTRICT 또는 ON DELETE CASCADE 등의 조건을 지정할 수 있다.

[리뷰(review) 테이블]

- 리뷰 테이블은 review_id를 기본키로 하며, user_id(FK), movie_id(FK), rating(INT CHECK(rating BETWEEN 1 AND 5)), comment(TEXT), created_at(DATE) 속성으로 구성된다.
- user_id는 사용자 테이블의 user_id를 참조하며, movie_id는 영화 테이블의 movie_id를 참조한다.
- 참조 무결성 조건으로 두 외래키는 각각 ON DELETE CASCADE, ON UPDATE CASCADE가 적용될 수 있어, 사용자가 삭제되거나 영화가 변경될 때 리뷰 정보가 자동으로 반영되거나 삭제된다.
- 리뷰는 사용자와 영화에 각각 N:1 관계를 가진다.

[배우(actor) 테이블]

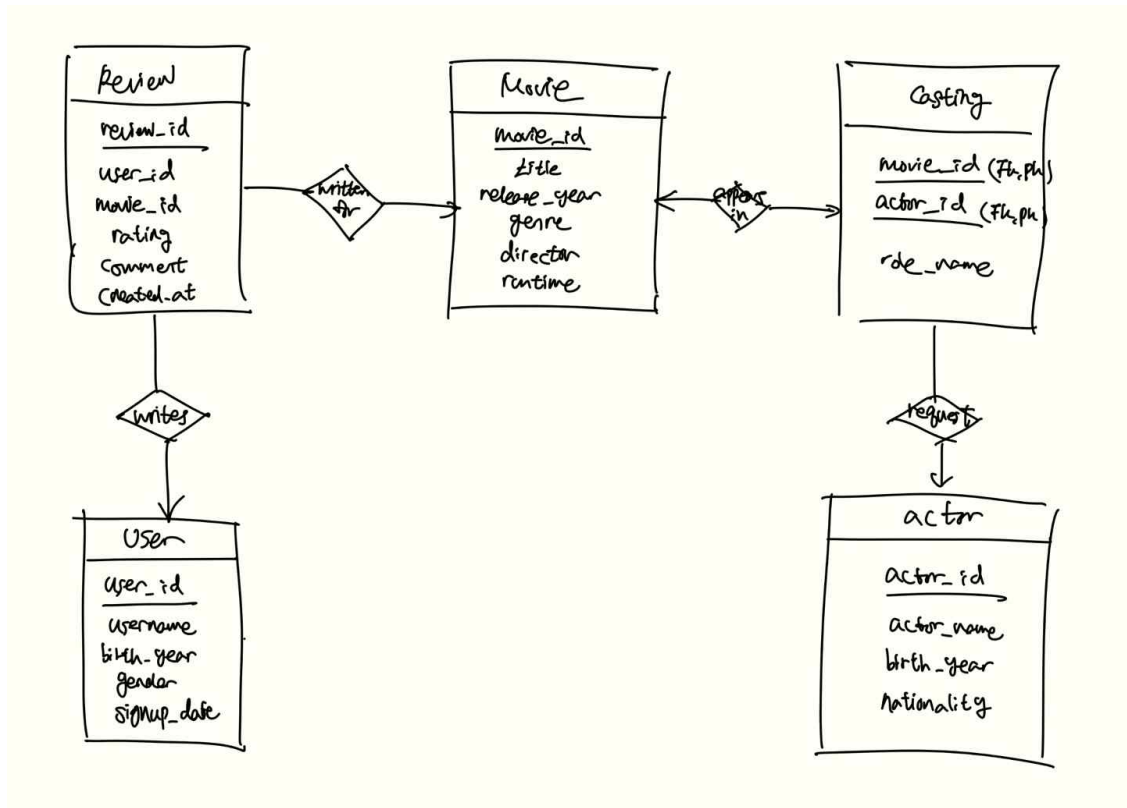
- 배우 테이블은 actor_id를 기본키로 하며, actor_name(VARCHAR, NOT NULL), birth_year(INT), nationality(VARCHAR) 속성으로 구성된다.
- 하나의 배우는 여러 영화에 출연할 수 있으므로 casting 테이블과 1:N 관계를 가진다.
- actor_id는 casting 테이블의 외래키로 참조되며, 참조 무결성을 보장하기 위해 ON DELETE SET NULL 또는 ON DELETE CASCADE가 설정될 수 있다.

[출연(casting) 테이블]

- casting 테이블은 영화와 배우 간의 다대다 관계(N:M)를 표현하는 연결 테이블로, movie_id(FK), actor_id(FK), role_name(VARCHAR) 속성으로 구성되며, movie_id와 actor_id를 합쳐 복합 기본키(PK)로 설정한다.
- movie_id는 movie 테이블의 기본키를 참조하고, actor_id는 actor 테이블의 기본키를 참조한다.
- 참조 무결성 조건으로 두 외래키 모두에 대해 ON DELETE CASCADE, ON UPDATE CASCADE가 적용될 수 있어, 영화나 배우가 삭제되면 자동으로 해당 출연 정보도 삭제된다.

정규화를 3NF까지 적용하여 각 테이블이 하나의 주제에 집중하도록 분리함으로써 데이터 중복을 최소화하고, 삽입·수정·삭제 시 발생할 수 있는 이상 현상(anomaly)을 방지하였음. 특히 영화, 사용자, 리뷰, 배우 등의 정보를 개별 테이블로 분리하면 데이터 무결성을 보장하면서도 다양한 조인을 통한 분석이 용이해짐.

3. ERD



4. SQL 정의 (DDL&DML)

```
CREATE TABLE user (  
    user_id INT PRIMARY KEY,  
    username VARCHAR(100) NOT NULL,  
    birth_year INT,  
    gender CHAR(1) CHECK (gender IN ('M','F')),  
    signup_date DATE  
);
```

```
CREATE TABLE movie (  
    movie_id INT PRIMARY KEY,  
    title VARCHAR(200) NOT NULL,  
    release_year INT,  
    genre VARCHAR(50),  
    director VARCHAR(100),  
    runtime INT  
);
```

```
CREATE TABLE review (  
    review_id INT PRIMARY KEY,  
    user_id INT,  
    movie_id INT,  
    rating INT CHECK (rating BETWEEN 1 AND 5),  
    comment TEXT,  
    created_at DATE,  
    FOREIGN KEY (user_id) REFERENCES user(user_id),  
    FOREIGN KEY (movie_id) REFERENCES movie(movie_id)  
);
```

```
INSERT INTO user VALUES (1, 'Rebecca', 2010, 'M', 2025-02-07);
```

```
INSERT INTO user VALUES (2, 'Sharon', 1971, 'F', 2022-12-27);
```

```
INSERT INTO user VALUES (3, 'Kathleen', 1985, 'M', 2024-05-29);
```

5. 샘플 데이터 생성

프롬프트 :

영화 리뷰 추천 시스템용 SQL 샘플 데이터를 생성해주세요.

총 5개의 테이블(user, movie, review, actor, casting)에 대해 다음 조건을 만족하는 데이터를 만들어주세요.

각 항목은 SQL INSERT 문으로 바로 사용할 수 있도록, 문자열은 따옴표(')로 감싸고 날짜 형식은 YYYY-MM-DD 형식으로 표기해주세요.

1. 사용자(user) 테이블 - 30명

- 속성: user_id (INT), username (VARCHAR), birth_year (INT), gender (CHAR), signup_date (DATE)
- 요구: user_id는 1~30의 정수, username은 실제 이름처럼, birth_year는 1970~2010 사이, gender는 'M' 또는 'F', signup_date는 최근 3년 이내 날짜

2. 영화(movie) 테이블 - 50편

- 속성: movie_id (INT), title (VARCHAR), release_year (INT), genre (VARCHAR), director (VARCHAR), runtime (INT)
- 요구: 다양한 장르(Action, Comedy, Drama 등), 감독 이름은 가상의 이름, 상영시간(runtime)은 80~160분, release_year는 2000~2024

3. 리뷰(review) 테이블 - 100건 이상

- 속성: review_id (INT), user_id (FK), movie_id (FK), rating (INT), comment (TEXT), created_at (DATE)
- 요구: rating은 1~5, comment는 영화 후기 느낌의 문장, created_at은 최근 1년 이내 날짜

4. 배우(actor) 테이블 - 40명

- 속성: actor_id (INT), actor_name (VARCHAR), birth_year (INT), nationality (VARCHAR)
- 요구: 다양한 국가 이름 포함, 이름은 실제 인물처럼 자연스럽게

5. 출연(casting) 테이블 - 영화당 3~5명 배우

- 속성: movie_id (FK), actor_id (FK), role_name (VARCHAR)
- 요구: role_name은 배우의 역할을 나타내는 직업 또는 역할명, 각 영화는 최소 3명 이상, 최대 5명 배우 출연

- 각 테이블은 기본키 제약조건을 지켜야 하며, review와 casting 테이블의 외래키 참조관계가 올바르게 연결될 수 있도록 user_id, movie_id, actor_id 값이 실제 존재하는 값이어야 합니다.

```
73 • INSERT INTO user VALUES (1, 'Michael', 2010, 'M', '2023-08-26');
74 • INSERT INTO user VALUES (2, 'Jason', 1971, 'F', '2022-06-28');
75 • INSERT INTO user VALUES (3, 'Ashley', 1985, 'M', '2024-11-18');
76 • INSERT INTO user VALUES (4, 'Russell', 1978, 'M', '2023-03-28');
77 • INSERT INTO user VALUES (5, 'Daniel', 2004, 'M', '2023-03-10');
78 • INSERT INTO user VALUES (6, 'Jennifer', 2007, 'F', '2024-09-22');
79 • INSERT INTO user VALUES (7, 'Duane', 1972, 'M', '2024-06-06');
80 • INSERT INTO user VALUES (8, 'Douglas', 1975, 'M', '2024-04-16');
81 • INSERT INTO user VALUES (9, 'Elijah', 1984, 'M', '2023-08-24');
82 • INSERT INTO user VALUES (10, 'Elizabeth', 2005, 'M', '2022-08-21');
83 • INSERT INTO user VALUES (11, 'Diana', 2004, 'F', '2023-03-17');
84 • INSERT INTO user VALUES (12, 'Amy', 1984, 'F', '2023-04-13');
85 • INSERT INTO user VALUES (13, 'Nicholas', 2007, 'F', '2022-11-08');
86 • INSERT INTO user VALUES (14, 'Justin', 1970, 'M', '2023-08-28');
87 • INSERT INTO user VALUES (15, 'Thomas', 1997, 'F', '2022-07-23');
88 • INSERT INTO user VALUES (16, 'Mario', 1987, 'M', '2024-08-17');
89 • INSERT INTO user VALUES (17, 'Paula', 1983, 'F', '2024-01-02');
90 • INSERT INTO user VALUES (18, 'Tiffany', 1976, 'M', '2023-12-28');
91 • INSERT INTO user VALUES (19, 'Nicole', 1994, 'M', '2024-11-06');
92 • INSERT INTO user VALUES (20, 'Laurie', 1992, 'F', '2024-04-18');
93 • INSERT INTO user VALUES (21, 'Ronald', 2008, 'F', '2022-12-19');
```

```

245 • INSERT INTO review VALUES (53, 8, 13, 4, 'Try result just data official out one catch.', '2024-09-12');
246 • INSERT INTO review VALUES (54, 12, 20, 2, 'Kind agent how argue modern type something represent whom land others attack million.', '2025-01-10');
247 • INSERT INTO review VALUES (55, 8, 2, 2, 'Among rule quality million consider mention heart public contain trouble enjoy drive.', '2025-01-25');
248 • INSERT INTO review VALUES (56, 13, 22, 3, 'War enjoy base care way new road culture market.', '2025-02-20');
249 • INSERT INTO review VALUES (57, 28, 5, 3, 'Responsibility nice campaign detail late class.', '2024-08-26');
250 • INSERT INTO review VALUES (58, 12, 42, 5, 'Minute speech available direction threat hot so according.', '2025-05-01');
251 • INSERT INTO review VALUES (59, 13, 44, 5, 'Full so loss detail which law pay.', '2025-01-20');
252 • INSERT INTO review VALUES (60, 11, 2, 1, 'Responsibility writer if avoid bag question art message skill.', '2024-11-15');
253 • INSERT INTO review VALUES (61, 29, 17, 2, 'Ask big tough car prove half involve pass change gun before five forget.', '2025-04-16');
254 • INSERT INTO review VALUES (62, 19, 17, 1, 'Stuff Mrs research happen drop article agency someone pattern treat form figure.', '2024-10-24');
255 • INSERT INTO review VALUES (63, 4, 39, 4, 'Watch quite together involve much better American.', '2025-05-19');
256 • INSERT INTO review VALUES (64, 12, 47, 3, 'Pull statement discussion drop dinner beyond will sure peace.', '2024-11-23');
257 • INSERT INTO review VALUES (65, 14, 39, 5, 'News technology support enough exactly official majority first without official.', '2024-07-16');
258 • INSERT INTO review VALUES (66, 4, 25, 5, 'Mother receive among then human life paper wear skill.', '2025-05-14');
259 • INSERT INTO review VALUES (67, 7, 17, 1, 'Then hot who range expect simply need third easy cost record read world.', '2024-06-04');
260 • INSERT INTO review VALUES (68, 23, 28, 1, 'Threat director reach deal high decade real matter technology recognize those within.', '2024-11-28');
261 • INSERT INTO review VALUES (69, 17, 35, 2, 'Toward couple front animal yes draw partner where I less treatment.', '2025-04-16');
262 • INSERT INTO review VALUES (70, 12, 28, 1, 'Exactly debate window recognize really various democratic study.', '2025-06-13');
263 • INSERT INTO review VALUES (71, 22, 22, 5, 'Real chance environment up anyone offer it strong.', '2025-05-16');
264 • INSERT INTO review VALUES (72, 11, 43, 1, 'Gun board seek most but any artist yet song realize.', '2025-01-23');
265 • INSERT INTO review VALUES (73, 24, 20, 5, 'Expect report north include majority dark site property.', '2024-12-23');
266 • INSERT INTO review VALUES (74, 10, 43, 4, 'Employee his quality major garden speak can build.', '2024-10-02');
267 • INSERT INTO review VALUES (75, 11, 26, 3, 'Poor green early imagine help less line appear church decide ability us wear.', '2025-06-07');

```

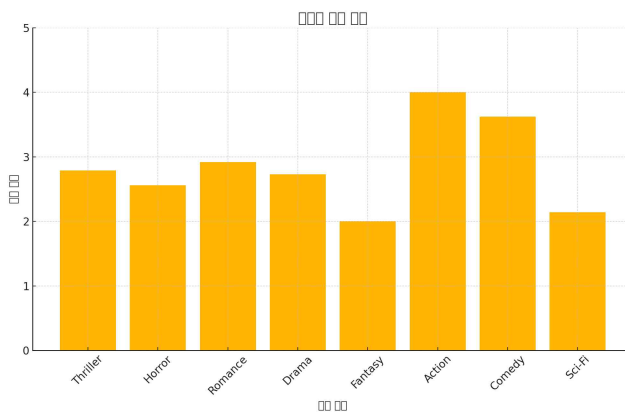
6. 데이터 분석 및 인사이트

- 장르별 평균 평점은 ?

```

SELECT genre, ROUND(AVG(rating), 2) AS avg_rating
FROM movie m
JOIN review r ON m.movie_id = r.movie_id
GROUP BY genre;

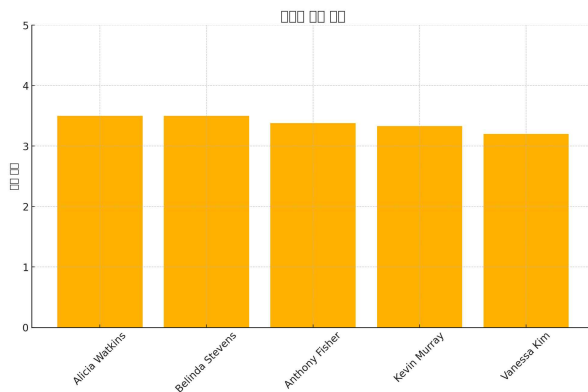
```



인사이트: 액션 장르는 평균 4점으로 가장 높고, 판타지 장르는 2점으로 낮게 나타남. 이는 관객들이 평균적으로 장르별 선호하는 정도가 다르다고 판단할 수 있다. 여름철의 경우 시원한 액션 장르의 인기가 일시적으로 높아진 것일 수 있으니, 지속적인 DB 분석을 통해 관객의 선호 장르를 파악해야 할 것이다.

- 감독별 평균 평점 상위 Top 5는 ?

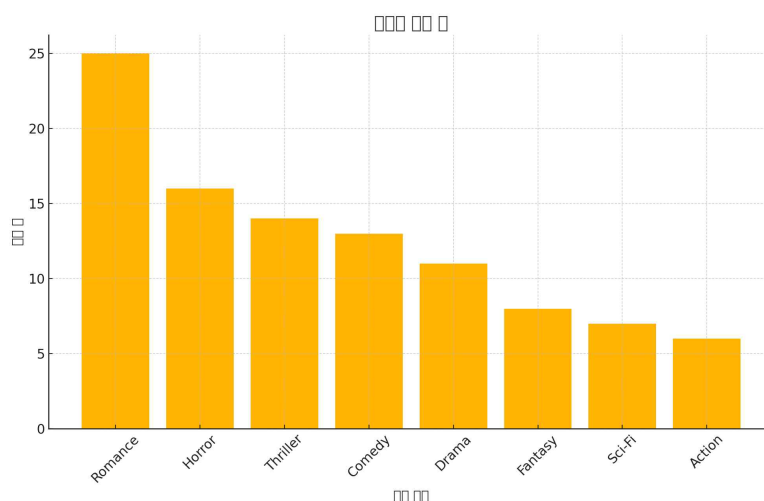
```
SELECT director, ROUND(AVG(r.rating), 2) AS avg_rating
FROM movie m
JOIN review r ON m.movie_id = r.movie_id
GROUP BY director
HAVING COUNT(*) >= 3
ORDER BY avg_rating DESC
LIMIT 5;
```



인사이트: 3편 이상 연출한 감독 중 Alicia Watkins, Belinda Stevens 감독의 영화가 평균 3.5 점으로 높은 평가를 받았다. 하지만 다른 감독들과의 격차가 크편이 아니기 때문에, 탭티어 감독이 연출한 경우, 감독이 누구인가는 관객 동원에 큰 영향을 끼치는 요인이 아닐 수 있다.

- 어떤 영화 장르가 리뷰 수가 가장 많은가 ?

```
SELECT m.genre, COUNT(r.review_id) AS review_count
FROM movie m
JOIN review r ON m.movie_id = r.movie_id
GROUP BY m.genre
ORDER BY review_count DESC;
```



인사이트: 리뷰 수 기준으로 가장 활발한 장르는 로맨스이며, 반면 판타지와 같은 특정 장르는 리뷰 수가 적어 관객 참여율이 낮은 경향을 보인다. 이는 장르별 마케팅 전략 수립에 참고하여 활용 가능하다. 1번 질문에서 장르별 평점이 가장 높은 장르가 액션이었는데, 표본 리뷰 수가 적어서 평점이 높게 나타난 것일 수 있으니, 결과 해석 시 주의가 필요하다.